04. Technical Proposal — EduAirControl

Version: 1.0
Date: 9 de octubre de 2026
Status: Technical proposal based on the repository and the available requirements specification.
Project: EduAirControl

> **Reading criterion:** this document distinguishes between the architecture described as implemented in `Docs-ea-control/05-architecture/overview.md` and recommendations or decisions that require validation. The presence of a component in the documentation is not, by itself, evidence that it is deployed and operational in all environments.

1. Executive summary

EduAirControl is a web and mobile solution to manage classroom and sensor information and to query environmental variables —temperature, humidity, CO_2 and noise level— in educational spaces. The technical proposal relies on a microservices architecture with an API gateway, REST/HTTP communication for synchronous operations and RabbitMQ for asynchronous events. PostgreSQL is used for persistence, Redis for cross-cutting gateway capabilities, and Docker Compose to standardize service execution.

The documented architecture also describes a gradual migration from a legacy monolith using the Strangler Fig pattern. Therefore, technical work should prioritize clear contracts between services, explicit data ownership, consistent end-to-end security, observability, and reproducible deployments.

This proposal organizes the solution, recommends implementation measures and highlights decisions that must be confirmed before considering the architecture production-ready.

2. Objective of the proposal

Define a technical basis to implement, integrate, test, deploy and maintain EduAirControl, with attention to:

- separation of responsibilities by domain;
- interoperability between client applications and services;
- protection of identity, permissions and data;
- reliable reception of sensor measurements;
- controlled evolution from the legacy system;

- maintainability, testing and operation.

3. Technical scope

3.1 Included

- React web application and React Native/Expo mobile application.
- Spring Cloud Gateway as the entry point for backend requests.
- Security, user management, classrooms, sensors, environmental monitoring and user experience services.
- PostgreSQL as the relational engine.
- RabbitMQ for publishing and consuming domain events.
- Redis for cross-cutting functions described in the architecture.
- HTTP ingestion from devices, with `X-API-Key` per device, according to the documented architecture.
- Docker containers and development orchestration via Docker Compose.
- Tests, environment configuration, API documentation and security practices.

3.2 Out of scope for this proposal

- Purchase, physical installation or certified calibration of sensors.
- Ensuring metrological accuracy without manufacturer specifications and calibration procedures.
- Multi-region high availability or a specific cloud infrastructure, since no target provider has been confirmed.
- Presenting MQTT as an implemented mechanism: the consulted architecture documents HTTP ingestion; any use of MQTT requires additional decisions and evidence.
- Declaring that all services are production-ready without a code, configuration and deployment audit.

4. Proposed technical architecture

4.1 Style and principles

The documented style is **microservices + API Gateway + event-based communication**. Migration is described as incremental, from a legacy monolith toward services bounded by context.

Recommended principles:

1. **Single responsibility per service:** each service owns its domain logic and data.
2. **Explicit contracts:** APIs and events are versioned and documented.
3. **Data ownership:** a service must not query another service's internal tables directly.
4. **Defense in depth:** authentication at the entry point and authorization also in the owning service.
5. **Resilience:** timeouts, limited retries, idempotency and failure handling.
6. **Externalized configuration:** secrets and parameters are supplied via secure configuration, not hard-coded in the repository.
7. **Traceability:** requests and events must be correlated using identifiers.
8. **Gradual evolution:** remove monolith routes only after validating functional and data equivalence.

4.2 Logical view of components

Component	Technical responsibility	Documented technology
Web client	Administrative interface and information query	React
Mobile client	Access from mobile devices	React Native / Expo
API Gateway	Backend entry point, routing, JWT validation, rate limiting and correlation identifiers	Spring Cloud Gateway
`ms-security`	Issuance and management of identity/tokens	Spring Security, JWT, OAuth2/OIDC described
`ms-user-management`	Management of the users domain	Independent microservice
`ms-classroom-management`	Management of classrooms	Independent microservice
`ms-sensor-management`	Management and registration of sensors	Independent microservice
`ms-environment-monitoring`	Reception/query of measurements and monitoring logic; absorbs the retired alerts service	Microservice + events
`ms-user-experience`	User experience domain functions	Independent microservice
PostgreSQL	Relational persistence	PostgreSQL
RabbitMQ	Asynchronous distribution of events	Topic exchange and idempotent consumers described
Redis	Rate limiting, sessions, revocation list and lock counters	Redis
Docker Compose	Coordinated local execution	Docker

Validation note: the architecture declares six PostgreSQL instances, but indicates that `ms-environment-monitoring` and `ms-user-experience` still use the `eduaircontrol` database with separate schemas. It is recommended to retain this situation as identified technical debt and decide on a complete separation with verified migrations, without assuming physical isolation already exists for all services.

4.3 Communication

Synchronous: client → API Gateway → responsible service, via REST/HTTP. In production, external traffic must use HTTPS. Internal routes and ports should remain on the private service network whenever possible.

Asynchronous: services publish events using a transactional outbox pattern and RabbitMQ; consumers must be idempotent because delivery may occur more than once. Each event should include a unique identifier, type, schema version, creation date, producer and correlation.

IoT devices: the consulted documentation defines HTTP ingestion with an `X-API-Key` header per device. Device identity, format, plausible physical limits, timestamp, duplicates and sending frequency must be validated. The key must be rotatable and revocable. The API key should not be treated as a substitute for transport encryption.

5. Technologies and justification

Layer	Documented base technology	Justification / criteria
Web	React	Component-based web interface
Mobile	React Native / Expo	Reuse of mobile capabilities and Expo development workflow
Backend	Spring Cloud Gateway and microservices from the Spring ecosystem described in the repository	Routing, security integration and domain separation
Identity	Spring Security, JWT and references to OAuth2/OIDC	Authentication and authorization; requires standardizing the scheme between gateway and services
Persistence	PostgreSQL	Relational model and transactional consistency
Messaging	RabbitMQ	Decoupling of events between domains
Cache/cross-cutting state	Redis	Rate limiting, sessions and documented lists/counters for security
Containers	Docker / Docker Compose	Repeatable environments for development and integration
Sensor ingestion	HTTP with `X-API-Key`	Mechanism documented in the current architecture

Exact versions of all components, support policies and base images must be fixed in build files and reviewed before releasing a version.

6. Service design and domain boundaries

6.1 Proposed catalog

- Security (`ms-security`): authentication, token issuance and identity-related capabilities.
- Users (`ms-user-management`): user creation, queries, changes and status.
- Classrooms (`ms-classroom-management`): classroom data, location, capacity and status.
- Sensors (`ms-sensor-management`): device registration, assignment, identification and status.
- Environmental monitoring (`ms-environment-monitoring`): ingestion and query of readings, evaluation against configured thresholds and eventual alert management.
- User experience (`ms-user-experience`): user experience domain functions, according to the actual repository contracts.

The names describe the boundaries shown in the architecture. Final responsibility for each endpoint must be confirmed against the code and the API catalog; business rules must not be duplicated across services.

6.2 Data ownership

Each service must own its entities and expose data to other services via APIs or events. Cross-service reads should not be performed directly against foreign tables. For the current shared database state, it is recommended to:

1. document which schema and tables belong to each service;
2. restrict database credentials per service;
3. measure existing dependencies before separating them;
4. migrate data with validation of counts, integrity and rollback capability;
5. remove legacy permissions and connections after verifying the change.

6.3 Environmental data

Each reading should identify, at minimum, the device or sensor, the variable, the value and unit, the measurement timestamp, the reception timestamp and the classroom or location reference. The exact structure must match the real data model and the project APIs.

Temperature, humidity, CO_2 and noise thresholds must be configurable and include source, unit, scope of application and version. The ranges in the requirements specification must not be

presented as universal standards nor substitute local regulations, professional guidance or device calibration.

7. Security

7.1 Identity and authorization

The architectural documentation describes that `ms-security` issues JWT RS256, while some services validate HS256 or trust internal headers such as `X-User-Id`, `X-User-Role` and `X-Institution-Id` during an intermediate stage. This difference is a consistency risk.

Priority recommendation: define and approve a single identity contract between gateway and services. At minimum:

- document algorithm, issuer, audience, expiration, keys and rotation;
- validate signature, `iss`, `aud`, `exp` and other required claims;
- avoid accepting identity headers from external clients;
- allow the gateway to strip and regenerate trusted internal headers;
- maintain role and institutional-scope authorization in the owning service;
- test $^{401}\!/_{403}$ cases and cross-institution access;
- design token revocation, expiration and renewal.

Do not change token algorithm or format without coordinated migration and compatibility testing.

7.2 Secrets and configuration

- Do not include real secrets in Git, images, public documentation or example files.
- Provide `.env.example` with fictitious values and validation of required variables.
- Use separate secrets per environment with least privilege.
- Rotate credentials upon exposure and document the procedure.
- Do not use insecure default values in production.
- Separate development, testing and production configuration.

7.3 API and device protection

- Apply rate limits and maximum request sizes.
- Validate data at the boundary and also in the owning service.

- Log authentication attempts and relevant events without exposing passwords, tokens or keys.
- Use unique API keys per sensor, with secure storage, rotation and revocation.
- Add replay or duplicate protection in ingestion where applicable.
- Review CORS to allow only known origins in each environment.
- Keep dependencies and images updated and analyze vulnerabilities.

8. Deployment and infrastructure

8.1 Development and integration

Docker Compose is the documented local execution tool. Configuration should include services, networks, persistent volumes for data that must be retained, health checks and startup dependencies. A service must not be considered ready solely because the container has started; it must pass its health check.

8.2 Environments

Environment	Use	Minimum requirements
Development	Individual work and early integration	Dummy data, useful logs, local configuration
Test/CI	Reproducible validation of changes	Automated tests, isolated databases, ephemeral configuration
Preproduction	Deployment and acceptance rehearsal	Configuration similar to production, without unnecessary sensitive data
Production	Real use	HTTPS, managed secrets, backups, monitoring, access control and proven recovery

The concrete production deployment strategy must still be selected. No specific cloud provider is proposed without budget, availability, data residency and operational requirements.

8.3 Recommended pipeline

1. Validate format, lint and build.

2. Run unit tests.
3. Run integration tests with isolated dependencies.
4. Validate REST contracts and events.
5. Analyze dependencies and containers.
6. Build immutably tagged images.
7. Deploy to test environment.
8. Run health checks and smoke tests.
9. Approve promotion to preproduction/production.
10. Maintain a rollback procedure.

This pipeline is a recommendation; it is not claimed that all its steps are already implemented.

9. Quality, testing and technical acceptance

A balanced test pyramid is recommended:

- Unit: domain rules, validations and transformations.
- Integration: persistence, migrations, security and messaging.
- Contract: compatibility of APIs and events between producer and consumer.
- End-to-end: critical flows from web/mobile to the services.
- Resilience: RabbitMQ/Redis outages, timeouts, retries and recovery.
- Security: authorization, secret exposure, API abuse and device onboarding.

Minimum technical acceptance criteria

- Services start with documented configuration and health checks.
- Endpoints have documented contracts, consistent error codes and input validation.
- Protected routes reject unauthenticated users and unauthorized roles.
- Events tolerate duplicates without producing duplicate business effects.
- Database migrations run reproducibly.
- Sensor readings are validated and traceable back to the device.
- Logs allow correlating a request between gateway and services.
- A backup exists and restoration has been tested.

- Critical dependencies and vulnerabilities are reviewed before release.

Quantitative thresholds for performance and availability must be defined based on expected load and institutional needs; no arbitrary figures are set in this proposal.

10. Observability and operation

The architecture mentions correlation identifiers and cross-cutting components, but the complete deployment of an observability platform is not confirmed. It is recommended to define:

- structured logs with `traceId` / `correlationId`, service, environment and outcome;
- metrics for latency, error rate, traffic, saturation, queues and database health;
- alerts for sustained failures, message accumulation, absence of sensor data and authentication errors;
- operational dashboards and incident response guides;
- retention and access policies for logs, avoiding sensitive data;
- service level objectives (SLOs) approved by responsible parties.

11. Recommended implementation plan

Phase	Activities	Verifiable outcome
1. Confirmation	Inventory of services, APIs, variables, dependencies and gaps between documentation and code	Matrix of real state and pending decisions
2. Foundations	Security contracts, configuration, health checks and API standards	Approved configuration and contracts
3. Main domains	Validation of users, classrooms and sensors; persistence and migrations	Management flows tested
4. Monitoring	HTTP ingestion, reading validation, threshold rules and events	Traceable readings and idempotency tests
5. Client integration	Connect web and mobile to gateway routes and test session errors	Critical client flows validated
6. Quality and security	Automated tests, dependency review, permissions and secrets	Results report and corrections
7. Operational readiness	Backups, restoration, metrics, manuals and rollback	Deployment readiness checklist
8. Gradual release	Deploy first to a controlled environment, compare results and migrate routes	Acceptance evidence and rollback plan

Phases are a suggested sequence; duration and final order must be adjusted to the real state of the repository and the team.

12. Risks and mitigations

Risk	Impact	Proposed mitigation
Different JWT schemes between gateway and services	Rejections, inconsistent permissions or route exposure	Single contract, integration tests and coordinated migration
Services sharing a physical database	Coupling and riskier deployments	Ownership by schema, minimal credentials and separation plan
Generic or outdated documentation	Decisions based on incorrect assumptions	Verify each claim against code, configuration and tests
Duplicate or out-of-order events	Repeated data or effects	Idempotency, unique identifiers and retry strategy
Exposed device credentials	Unauthorized ingestion	Unique keys, rotation, revocation and encrypted transport
Environmental thresholds without contextual validation	Inadequate alerts	Configurable thresholds, documented source and institutional approval
Lack of proven recovery	Prolonged service/data loss	Backups and restoration drills
Incomplete monolith migration	Duplicated or inconsistent functions	Route-by-route migration, comparison and rollback plan

13. Decisions requiring confirmation

Before finalizing the definitive design, the team must confirm:

1. The final JWT contract: RS256/JWKS versus HS256 validation and use of internal headers.
2. The real state and versions of each service and dependency.
3. The strategy for separating databases and shared schemas.
4. The definitive API and event contracts, including versioning and compatibility.

5. The official source and configuration of environmental thresholds.
6. The production deployment environment, expected availability and data residency.
7. Measurable objectives for performance, retention, recovery and availability.
8. Whether MQTT will be required in the future; it must not be assumed implemented with current evidence.
9. Institutional and legal requirements applicable to processing user data and measurements.

14. Derived technical deliverables

- Updated C4 context and container diagram.
- Catalog of microservices, owners and dependencies.
- Versioned REST and event contracts.
- Data model and ownership matrix by service.
- Reproducible development and CI configuration.
- Automated test suite and results report.
- Deployment, rollback, backup and recovery guide.
- Operations, monitoring and incident response manual.
- Risk matrix and architecture decisions (ADR).

Budget and development time

15. Project development time

The EduAirControl project had an approximate working period of **18 months**, during which analysis, design, development, integration, testing and documentation activities were carried out.

However, it is estimated that the project could be developed in a period of **4 to 5 months of intensive work**, considering an 8-hour daily dedication. This difference is because the 18 months correspond to the elapsed time during the project preparation and do not necessarily reflect continuous full-time work.

Concept	Estimation
Time elapsed during the project	18 months
Estimated intensive development time	4 a 5 meses
Reference working day	8 horas diarias
Estimated cost of software development	$20.000.000 COP

The development cost represents an estimate of the effort for analysis, programming, integration, testing and documentation. It does not necessarily correspond to money actually paid, so it should be presented as an estimated cost if the team did not receive that remuneration.

16. IoT hardware budget

To develop and test the environmental monitoring component, two computers and the electronic devices necessary to take measurements of the environmental variables are contemplated.

Category	Quantity	Unit price	Total cost
Laptops	2	$5.500.000	$11.000.000
Mouse	2	$160.000	$320.000
ESP32 development board	1	$40.000	$40.000
DHT11 temperature and humidity sensor	1	$8.000	$8.000
KY-038 sound sensor/module	1	$8.000	$8.000
MQ-135 air quality sensor	1	$14.000	$14.000
Total hardware			**$11.390.000**

Technical observation: the DHT11 measures temperature and relative humidity. The KY-038 detects sound and can provide a relative level through its analog output; it should not be interpreted as a calibrated sound-level meter in dB without a calibration procedure. The MQ-135 can detect changes in the concentration of certain gases, but does not by itself provide a precise and selective CO_2 measurement in ppm. Therefore, hardware specifications must match the devices actually used.

17. Electricity and internet expenses

During the 18 months of work, electricity and internet expenses necessary for programming, testing, consulting documentation, managing the repository and connecting the devices are also considered.

As the real invoices are not available here, the following reference budget is used:

Concept	Estimated monthly cost	Period	Estimated cost
Electricity attributable to the project	$80.000	18 months	$1.440.000
Internet service	$120.000	18 months	$2.160.000
Total services	**$200.000**	**18 months**	**$3.600.000**

These values are budgetary estimates, not verified expenses. To present the real cost, they must be replaced with the amounts actually paid or with a reasonable proportion of the invoices attributable to the project.

18. Overall budget

Integrating the estimated development cost, hardware and services used during the project yields the following budget:

Concept	Value
Software development	$20.000.000
Hardware for development and testing	$11.390.000
Electricity for 18 months	$1.440.000
Internet for 18 months	$2.160.000
Estimated total budget	**$34.990.000 COP**

19. Budget considerations

The estimated total budget for EduAirControl is **$34.990.000 COP**. This value includes the reference cost of software development, computer equipment, IoT electronic components and estimated electricity and internet expenses.

The budget should not be automatically interpreted as disbursed money. The development cost is a valuation of the work performed, while equipment and services may represent actual purchases or expenses if receipts are available.

Values may also vary according to market prices, the equipment ultimately purchased, energy consumption, the internet plan and the effective time dedication of the team.

20. Conclusion

The technical foundation of EduAirControl aims at a modular and extensible architecture that combines web/mobile clients, microservices, gateway, PostgreSQL, RabbitMQ, Redis and containers. To turn that foundation into a verifiably stable solution, priorities are to standardize the security contract, reinforce data ownership, document APIs/events, automate tests, validate sensor ingestion and finalize deployment and recovery aspects.

This proposal is a design and implementation guide based on the available artifacts. Decisions marked as pending must be resolved with evidence from code, tests and real environment needs before certifying the system as production-ready.

21. Internal sources consulted

- `Docs-ea-control/05-architecture/overview.md` — architecture, services, communication and technologies.
- `Docs-ea-control/05-architecture/README.md` — architecture decisions and ADR status.
- `Docs-ea-control/09-microservices/README.md` — organization of microservices documentation.
- `Docs-ea-control/12-ux-ui/design-system.md` — interface guidelines; treated as design guidance, not proof of implementation.
- `SRS_EduAirControl` provided by the team — system scope and requirements.